QueryGenie — Review-III speaker script

Team 21 · Supervisor: M Parameswar, Associate Professor · Deck: QueryGenie_Review-III.pptx (18 slides)

Who presents what

Speaker	Slides	Time
Jayakanth Kamisetti (team lead)	1–3 opening · 12–13 + live demo · 16–18 close; leads Q&A	≈ 5 min + 5 min demo
Boddupally Narendar	4–8 related papers, research gap · 14–15 limitations, future work	≈ 4 min
M Praneeth Reddy	9–11 datasets, architecture, the app	≈ 2 min

Order: Jayakanth → Narendar → Praneeth → Jayakanth (results, demo) → Narendar → Jayakanth (close).

Ground rules

- Say the numbers exactly as written here — they are the measured ones and match the deck and report.
- Never say we beat the paper. We matched it (77.8% vs 77.9%) and made it more trustworthy.
- Speak to the panel, not the slide. Glance at the slide only for the numbers.
- End your part with the handoff line, so the next speaker knows to start.
- If you don't know an answer in Q&A, say "Jayakanth will take that" — don't guess.

Part 1 · Opening — Jayakanth Kamisetti

Part 1 · Opening — slides 1–3 · about 1.5 minutes

Slide 1 · Title

Say: Good morning. I am Jayakanth Kamisetti, team lead of Team 21, with my teammates Boddupally Narendar and Praneeth Reddy. Our supervisor is M Parameswar, Associate Professor. Our project is QueryGenie: a self-correcting natural-language-to-SQL interface.

Slide 2 · Outline

Say: We will cover the problem, the research we built on, the gap we found, our method and data, the results — including a live demo — then the limitations and our plans.

Slide 3 · Problem Statement

Say: Most of an organisation's data lives in databases, and getting an answer out of them needs SQL. Text-to-SQL systems let you ask in plain English instead. The problem is that they answer every question with the same confidence, so a wrong answer looks exactly like a right one. Even CodeS, a strong open model, is wrong on about one in five questions of the standard Spider benchmark. And most strong systems use closed models like GPT-4, which means sending your database schema to a third party. Our goal: reproduce CodeS, an open model that runs locally, and make it say when it is unsure instead of answering wrongly. Narendar will now take you through the research we built on.

Part 2 · Related work — Boddupally Narendar

Part 2 · Related work — slides 4–8 · about 2.5 minutes

Slide 4–7 · Related Research Papers

Don't read the table — point at a few rows per slide.

Say: Thank you. We reviewed sixteen papers, all from arXiv. On this slide: Seq2SQL and Spider started the field — Spider has over ten thousand questions on two hundred databases, and the best model at the time got only 12.4 percent. Next slide: neural approaches like BRIDGE, which adds matching database values to the input, and PICARD, which forces the model to write valid SQL. Then the large-language-model era: DIN-SQL adds self-correction and reaches 85.3 percent on Spider's test set; BIRD shows that on large real-world databases even ChatGPT gets only about 40 percent, against 93 percent for humans. On the last slide: CHESS and CHASE-SQL, and CodeS — the paper we reproduce — open models from one to fifteen billion parameters; CodeS-1B reports 77.9 percent on Spider.

Slide 8 · Research Gap

Say: We found three gaps. First, none of these systems says when it is unsure — every answer comes with the same confidence. Second, the strongest ones depend on closed models. Third, results are reported on Spider, whose questions often name the columns; how the same model does on more realistic questions is rarely measured. Our idea: CodeS already writes four candidate queries for every question. If they agree, answer; if they disagree, ask the user. Praneeth will now explain our data and how the system works.

Part 3 · Method — M Praneeth Reddy

Part 3 · Method — slides 9–11 · about 2 minutes

Slide 9 · Proposed Methodology — datasets

Say: Thank you, Narendar. All our data comes from Hugging Face, which is on the department's approved list — no Kaggle. Our main benchmark is the Spider development set: 1,034 questions on 20 databases the model has never seen. About 40 percent of the questions need a join between tables. We also use Spider-Realistic: 508 of the same questions reworded so they don't mention the column names, the way real users talk. And we set up BIRD Mini-Dev, 500 questions on eleven large real-world databases.

Slide 10 · Proposed Methodology — architecture

Point at the three colours as you go.

Say: Here is the system. In blue is the CodeS pipeline we reproduced: for each question it builds a description of the database — tables, columns, keys, sample values, and values that match words in the question — and the model writes four candidate SQL queries. In amber is our enhancement: we run all four queries and count how many return the same answer. If enough agree, we answer; if not, we show the user the different readings and ask which one they meant. In green is the application that wraps it all.

Slide 11 · Proposed Methodology — the QueryGenie app

Say: QueryGenie is a desktop app for Mac and Windows, and it runs completely on the user's computer — nothing is sent anywhere. On the left, a confident answer: all four queries agree, so it shows the table, a chart and the SQL. On the right, the queries disagree, so it asks which reading you meant instead of guessing. Jayakanth will now show the results and a live demo.

Part 4 · Results and live demo — Jayakanth Kamisetti

Part 4 · Results and live demo — slides 12–13 + demo · about 3 minutes + 5-minute demo

Slide 12 · Experimental Results

Say: Thank you, Praneeth. First, the reproduction. With the paper's own official evaluator we get 77.8 percent execution accuracy on Spider, against 77.9 percent in the paper — within 0.1 points, on a free Colab GPU instead of the authors' eight A800s. The ablation shows every part of the prompt matters: removing any one costs 0.9 to 1.9 points. Then our enhancement, on the same 1,034 questions: it answers 90.4 percent of them, and on those it is right 83.3 percent of the time, against 77.8 percent. Wrong answers drop from 230 to 156 — about a third. The paired t-test gives p of 7.7 times ten to the minus ten, and the Wilcoxon test agrees. And the signal is calibrated: when only one of four queries agrees, it is right 57 percent of the time; when all four agree, 88 percent.

Slide 13 · Experimental Results (continued)

Say: This curve shows the trade-off: the fewer questions it answers, the more accurate it becomes. We also measured self-correction honestly: sending failed queries back to the model changes accuracy from 77.76 to 77.85 percent — CodeS was never trained to read error messages, which is why abstention became our main enhancement. Finally, the benchmark-to-reality gap: on Spider-Realistic we get 69.5 percent against the paper's 70.9 — a second reproduction point within tolerance — and removing column names from the questions costs 3.3 points. BIRD is set up but not completed. Now let me show it live.

Slide Live demo

Follow Demo_Script.pdf exactly — questions 1, 5, 6 and the Map are the essentials.

Say: Ask: 'List the subjects and the average marks for each subject.' — four of four agree, open the chart. Then: 'Who topped each subject?' — it says it is not sure and shows the readings: this is our enhancement. Then 'show me all' — it asks 'all of what?'. Then open the Map. Back to the slides: Narendar will cover the limitations and what comes next.

Part 5 · Limitations and future work — Boddupally Narendar

Part 5 · Limitations and future work — slides 14–15 · about 1.5 minutes

Slide 14 · Limitations

Say: Thank you. To be clear about the limits: overall accuracy is unchanged at 77.8 percent — we are right more often when we answer by declining about ten percent of questions. Agreement is not proof: about 12 percent of answers where all four agree are still wrong. The threshold was tuned on Spider with CodeS-1B. CodeS answers one question at a time, so follow-ups need a chat model. And we reproduced inference only, because pre-training needs eight A800 GPUs.

Slide 15 · Conclusion and Future Work

Say: In short: we reproduced CodeS within 0.1 points, made it say when it is unsure — 83.3 percent correct when it answers, a third fewer wrong answers, statistically significant — and delivered it as a local desktop app. Next, on the research side: CodeS-3B, a reranker — a correct query is among the four for 83 percent of questions — and calibration on real-world data. On the product side, we want to turn it into a platform companies can plug into any database, with access control and adaptation to each database, still running locally. Jayakanth will close.

Part 6 · Close and Q&A — Jayakanth Kamisetti

Part 6 · Close and Q&A — slides 16–18

Slide 16–17 · References and GitHub

Say: Our references are all from arXiv, and all our code, data processing and every logged run are on GitHub at github.com/jayakanthh/QueryGenie.

Slide 18 · Thank you

Say: Thank you. We are happy to take your questions — I'll direct each question to the right teammate.

Likely questions — who answers

Boddupally Narendar

- Why CodeS? — open weights, code and evaluation are public, so it can be checked and run locally; the paper reports state-of-the-art results on nearly all the benchmarks it tested.
- Where do the papers and data come from? — papers only from arXiv; data only from Hugging Face (Spider, Spider-Realistic, BIRD); no Kaggle.
- How is this different from ChatGPT? — ChatGPT answers everything with the same confidence and needs your schema; ours runs locally and says when it isn't sure.

M Praneeth Reddy

- How does the model see the database? — as text: tables, columns, keys, a few sample values, and stored values that match words in the question.
- Why four queries? — that's what CodeS already generates with beam search, in one run; we only execute and compare them.
- What databases does the app support? — SQLite files, PostgreSQL and MySQL (read-only); other models through Ollama or cloud services, marked as not calibrated.

Jayakanth Kamisetti

- Why not exactly 77.9%? — different hardware and library versions; measured: the same 508 questions scored 72.8% on the Mac and 73.2% on Colab. Within the ±5–10% tolerance.
- Did you beat the paper? — not on overall accuracy (still 77.8%); the improvement is trustworthiness: 83.3% correct when it answers, 230 → 156 wrong answers.
- Why not majority vote? — we tested it: it lowers accuracy to 75.7%. Agreement tells you whether to trust an answer, not which answer to give.
- How was the threshold chosen? — fixed in advance; set on random calibration halves for 90% coverage, tested on the other halves, 10 times.
- Self-correction is in the title — does it work? — it's in the app; measured on Spider it barely helps (77.76 → 77.85%) because CodeS can't use error messages.